Week #1: Introduction to Numerical Methods

Numerical Methods: Mathematical Foundations

June 7, 2025

Overview

Numerical methods form the mathematical foundation of scientific computing, providing systematic approaches to solve problems that cannot be solved analytically. This introductory week establishes the theoretical framework for understanding how continuous mathematical problems are approximated using discrete computational methods.

The central challenge in numerical analysis lies in the interplay between mathematical rigor and computational feasibility. We explore how approximation errors arise, propagate, and can be controlled through careful algorithm design. The concepts developed here—well-posed problems, stability, convergence, and computational complexity—form the theoretical backbone for all subsequent numerical methods.

Our approach emphasizes mathematical precision while building intuition for the practical considerations that guide algorithm selection and implementation. We establish the vocabulary and conceptual framework that will unify the diverse numerical methods studied throughout this course.

---

1 Mathematical Foundations of Scientific Computing

1.1 Well-Posed Problems

Definition. Well-Posed Problem

A mathematical problem is well-posed in the sense of Hadamard if it satisfies three conditions:

1. **Existence:** A solution exists
2. **Uniqueness:** The solution is unique
3. **Stability:** The solution depends continuously on the input data

The concept of well-posedness provides the foundation for numerical analysis. If a problem is not well-posed, numerical methods may fail catastrophically or produce meaningless results.

Example 1.1 (Well-Posed vs. Ill-Posed Problems)**.** ***Well-posed:*** *Solving $Ax = b$ where A is invertible. - Existence: Solution $x = A^{-1}b$ exists - Uniqueness: Matrix invertibility guarantees uniqueness - Stability: Small changes in b produce small changes in x (if A is well-conditioned)*

Ill-posed: *Numerical differentiation from noisy data. - Small measurement errors can lead to arbitrarily large errors in the computed derivative - The problem lacks stability in the sense of Hadamard*

Theorem 1.1 (Continuous Dependence on Data)**.** *For a well-posed problem with solution operator S, there exists a constant $C > 0$ such that:*

$$\|S(f_1) - S(f_2)\| \leq C\|f_1 - f_2\|$$

for input data f_1, f_2 in some neighborhood.

This ensures that small perturbations in input data lead to small changes in the solution.

1.2 Sources of Error in Numerical Computation

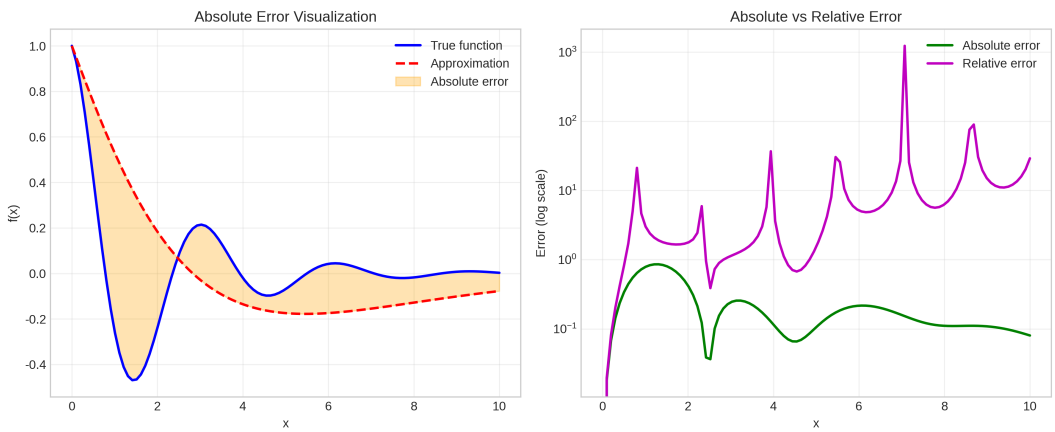

Figure 1: Visualization of absolute and relative errors in numerical approximation. The left panel shows how approximation error manifests as the difference between true and approximate functions. The right panel compares absolute and relative error behavior.

Numerical errors arise from multiple sources, each requiring different mathematical treatment:

Definition. Types of Numerical Error

- **Modeling Error:** Difference between physical reality and mathematical model
- **Discretization Error:** Error from replacing continuous problems with discrete approximations
- **Roundoff Error:** Error from finite precision arithmetic
- **Truncation Error:** Error from stopping infinite processes after finite steps

Definition. Absolute and Relative Error

For an exact value x and approximation $\hat{x}$:

$$\text{Absolute Error} = |x - \hat{x}| \tag{1}$$

$$\text{Relative Error} = \frac{|x - \hat{x}|}{|x|} \quad (x \neq 0) \tag{2}$$

Relative error provides scale-independent error measurement, crucial for comparing errors across different problem sizes.

Theorem 1.2 (Error Propagation in Arithmetic Operations)**.** *For operations on approximate values $\hat{a} = a + \epsilon_a$ and $\hat{b} = b + \epsilon_b$:*

Addition/Subtraction:

$$\hat{a} \pm \hat{b} = (a \pm b) + (\epsilon_a \pm \epsilon_b)$$

Absolute errors add directly.

Multiplication:

$$\hat{a} \cdot \hat{b} = ab + a\epsilon_b + b\epsilon_a + \epsilon_a\epsilon_b$$

Relative errors approximately add: $\frac{\epsilon_{ab}}{ab} \approx \frac{\epsilon_a}{a} + \frac{\epsilon_b}{b}$

Division:

$$\frac{\hat{a}}{\hat{b}} \approx \frac{a}{b}\left(1 + \frac{\epsilon_a}{a} - \frac{\epsilon_b}{b}\right)$$

Intuition: Understanding error propagation

Error propagation reveals why certain computations are more sensitive to input errors than others:

- Addition/subtraction: Absolute errors matter most
- Multiplication/division: Relative errors matter most
- Subtraction of nearly equal numbers: Can cause catastrophic cancellation
- Functions with large derivatives: Amplify input errors significantly

This understanding guides the design of numerically stable algorithms.

---

2 Stability and Conditioning

2.1 Forward and Backward Error Analysis

Definition. Forward and Backward Error

Consider a problem $f : X \to Y$ with exact solution $y = f(x)$ and computed solution $\hat{y}$.

Forward Error: $\|y - \hat{y}\|$ (error in the output)

Backward Error: $\min\{\|\Delta x\| : \hat{y} = f(x + \Delta x)\}$ (perturbation in input that would produce the computed output exactly)

Theorem 2.1 (Relationship Between Forward and Backward Error)**.** *If the backward error is $\|\Delta x\|$ and the problem has condition number κ, then:*

$$\textit{Forward Error} \leq \kappa \cdot \textit{Backward Error}$$

This fundamental relationship separates algorithm stability (backward error) from problem conditioning (amplification factor).

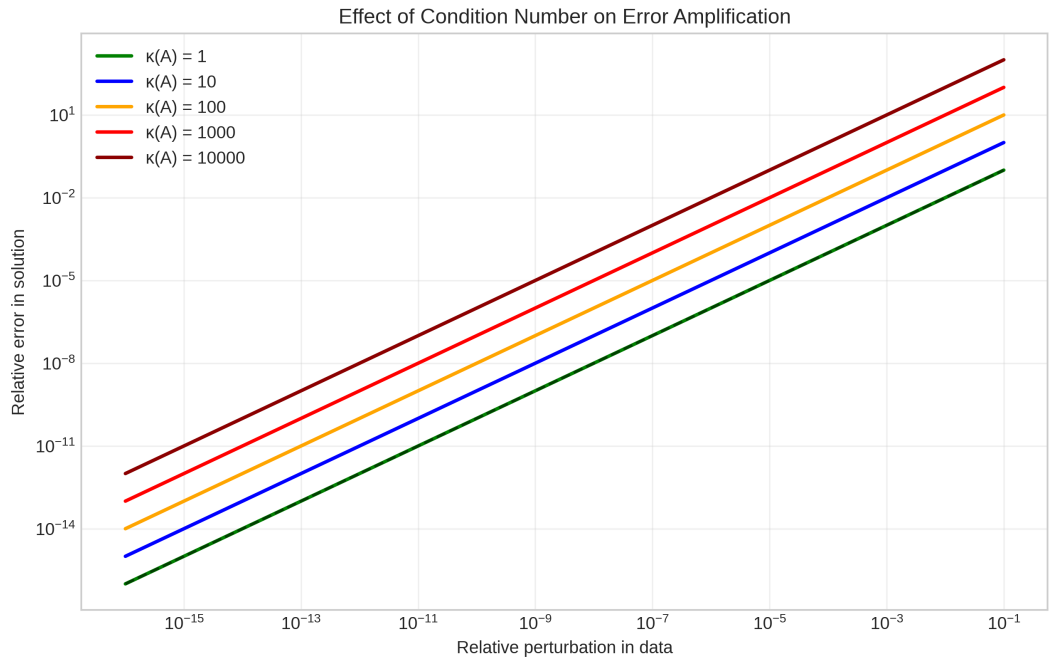

Figure 2: Effect of condition number on error amplification. Each line shows how relative perturbations in input data are amplified for different condition numbers. Well-conditioned problems ($\kappa \approx 1$) show minimal amplification, while ill-conditioned problems ($\kappa \gg 1$) can amplify errors dramatically.

2.2 Condition Numbers

Definition. Condition Number

The condition number of a problem measures the sensitivity of the solution to perturbations in the input data:

$$\kappa = \lim_{\epsilon \to 0} \sup_{\|\Delta x\| \leq \epsilon} \frac{\|f(x + \Delta x) - f(x)\| / \|f(x)\|}{\|\Delta x\| / \|x\|}$$

For differentiable functions: $\kappa = \frac{\|f'(x)\| \|x\|}{\|f(x)\|}$

Example 2.1 (Condition Numbers in Practice)**.** ***Linear Systems:*** *For $Ax = b$, the condition number is:*

$$\kappa(A) = \|A\| \|A^{-1}\|$$

Polynomial Evaluation: *For $p(x) = \sum_{i=0}^{n} a_i x^i$:*

$$\kappa = \frac{\sum_{i=0}^{n} |a_i x^i|}{|p(x)|}$$

Function Evaluation: *For $f(x)$:*

$$\kappa = \left| \frac{x f'(x)}{f(x)} \right|$$

Theorem 2.2 (Condition Number Properties)**.**

1. *$\kappa \geq 1$ for any nontrivial problem*
2. *$\kappa = 1$ indicates perfect conditioning*
3. *$\kappa \gg 1$ indicates ill-conditioning*
4. *For matrix problems: $\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$ where σ denotes singular values*

2.3 Numerical Stability

Definition. Numerical Stability

An algorithm is **numerically stable** if it produces a solution $\hat{y}$ that is the exact solution to a slightly perturbed problem:

$$\hat{y} = f(x + \Delta x)$$

where $\|\Delta x\|$ is small relative to $\|x\|$.

The algorithm is **backward stable** if $\|\Delta x\| = O(\epsilon_{\text{machine}})\|x\|$.

Theorem 2.3 (Stability vs. Accuracy)**.** *For a backward stable algorithm applied to a problem with condition number κ:*

$$\frac{\|\textit{Forward Error}\|}{\|\textit{Exact Solution}\|} \leq \kappa \cdot \epsilon_{\textit{machine}}$$

This shows that:

- *Stable algorithms control backward error*
- *Problem conditioning determines forward error amplification*
- *Accuracy = Stability × Conditioning*

Intuition: The stability principle

Numerical stability is about algorithm design, while conditioning is an inherent property of the mathematical problem:

- A stable algorithm applied to a well-conditioned problem yields accurate results
- A stable algorithm applied to an ill-conditioned problem may yield inaccurate results, but this reflects the problem's inherent sensitivity, not algorithmic failure
- An unstable algorithm can produce poor results even for well-conditioned problems

The goal is to design stable algorithms and recognize when problems are ill-conditioned.

---

3 Convergence and Computational Complexity

3.1 Convergence Analysis

Definition. Types of Convergence

For a sequence $\{x_k\}$ converging to x^*:

Linear Convergence: $\|x_{k+1} - x^*\| \leq C\|x_k - x^*\|$ for some $C < 1$

Quadratic Convergence: $\|x_{k+1} - x^*\| \leq C\|x_k - x^*\|^2$

Superlinear Convergence: $\lim_{k\to\infty} \frac{\|x_{k+1}-x^*\|}{\|x_k-x^*\|} = 0$

Theorem 3.1 (Convergence Rate Comparison)**.** *For achieving error tolerance ϵ:*

- *Linear convergence: $O(\log(1/\epsilon))$ iterations*

- *Quadratic convergence:* $O(\log\log(1/\epsilon))$ *iterations*
- *Superlinear convergence: Between linear and quadratic*

Quadratic convergence provides dramatic improvement once the iterates are sufficiently close to the solution.

3.2 Computational Complexity

Definition. Algorithm Complexity

Time Complexity: Number of elementary operations as a function of problem size n

Space Complexity: Memory requirements as a function of problem size n

Common complexity classes:

- $O(n)$: Linear algorithms
- $O(n \log n)$: Divide-and-conquer algorithms
- $O(n^2)$: Quadratic algorithms
- $O(n^3)$: Cubic algorithms (e.g., matrix multiplication)
- $O(2^n)$: Exponential algorithms

Example 3.1 (Complexity in Numerical Linear Algebra)**.** ***Matrix-vector multiplication:*** $O(n^2)$ *operations* ***Matrix-matrix multiplication:*** $O(n^3)$ *operations (naive algorithm)* ***LU decomposition:*** $O(n^3)$ *operations* ***Solving triangular systems:*** $O(n^2)$ *operations* ***Computing eigenvalues:*** $O(n^3)$ *operations (dense matrices)*

Theorem 3.2 (Complexity Trade-offs)**.** *Numerical algorithms often involve trade-offs:*

- ***Direct vs. Iterative:*** *Direct methods have predictable cost but may be expensive; iterative methods have variable cost but can be more efficient for large problems*
- ***Accuracy vs. Speed:*** *Higher accuracy often requires more computation*
- ***Memory vs. Time:*** *Storing intermediate results can reduce computation but increases memory requirements*

Practice Problems

1. Prove that the condition number of matrix multiplication satisfies $\kappa(AB) \leq \kappa(A)\kappa(B)$.
2. Show that evaluating $f(x) = x^n$ has condition number $\kappa = n$ for $x > 1$.
3. Analyze the stability of computing $(a + b) - (c + d)$ versus $(a - c) + (b - d)$ when $a \approx c$ and $b \approx d$.

Solution:

Problem 1: For matrices A and B, we have: $\kappa(AB) = \|AB\|\|(AB)^{-1}\| = \|AB\|\|B^{-1}A^{-1}\|$

Using submultiplicativity of matrix norms: $\|AB\| \leq \|A\|\|B\|$ and $\|B^{-1}A^{-1}\| \leq \|B^{-1}\|\|A^{-1}\|$

Therefore: $\kappa(AB) \leq \|A\|\|B\|\|A^{-1}\|\|B^{-1}\| = \kappa(A)\kappa(B)$

Problem 2: For $f(x) = x^n$, we have $f'(x) = nx^{n-1}$. The condition number is: $\kappa = \left|\frac{xf'(x)}{f(x)}\right| = \left|\frac{x \cdot nx^{n-1}}{x^n}\right| = |n|$

For $x > 1$, this gives $\kappa = n$.

Problem 3: Let $a = c + \epsilon_1$ and $b = d + \epsilon_2$ where ϵ_1, ϵ_2 are small.

Method 1: $(a + b) - (c + d) = (c + \epsilon_1 + d + \epsilon_2) - (c + d) = \epsilon_1 + \epsilon_2$

Method 2: $(a - c) + (b - d) = \epsilon_1 + \epsilon_2$

Both methods are algebraically equivalent and equally stable. The key insight is that when $a \approx c$ and $b \approx d$, both expressions involve subtraction of nearly equal quantities, which can amplify relative errors but doesn't create additional instability.

Summary and Looking Ahead

This introductory week has established the fundamental mathematical framework for numerical analysis. We have explored the concepts of well-posed problems, error analysis, stability, conditioning, and computational complexity. These concepts form the theoretical foundation that will guide our study of specific numerical methods throughout the course.

The key insights from this week include:

- Well-posedness is essential for meaningful numerical computation
- Error analysis requires distinguishing between different error sources and types
- Stability and conditioning are separate but related concepts that together determine accuracy
- Convergence analysis provides theoretical guarantees for iterative methods
- Computational complexity guides algorithm selection for different problem sizes

Next week, we will delve deeper into computer arithmetic and floating-point systems, examining how the finite precision of digital computers affects numerical computation. We will study the IEEE 754 standard, analyze roundoff error propagation, and develop strategies for writing numerically robust code.

The mathematical principles established this week will provide the foundation for understanding how theoretical error bounds translate into practical computational considerations.